

CSD 420 Advanced Java Programming

Module 11 Programming Assignment - Java JSON API Paper

Fernando Contreras

October 7, 2026

Jackson: A High-Performance JSON Processor for Java

Introduction and History

Jackson is one of the most widely used Java libraries for processing JSON data. It was originally created by Tatu Saloranta in 2007 and started as a streaming JSON parser called Jackson JSON Processor. Over time it grew into a complete suite of modules for working with JSON, XML, YAML, CSV, and other data formats. Today it is maintained as an open source project under the Apache 2.0 license and is the default JSON library used by major frameworks like Spring Boot, which means a huge percentage of Java backend applications in production rely on Jackson under the hood. The project is hosted on GitHub under the FasterXML organization and has been continuously developed for nearly two decades.

Features and Processes Supported

Jackson supports three main approaches to working with JSON, each suited to different use cases. The first is the streaming API, which reads and writes JSON one token at a time using `JsonParser` and `JsonGenerator`. This approach is the fastest and uses the least memory, which makes it a good fit for very large files or performance critical code where every millisecond matters. The second is the tree model, where JSON is parsed into a tree of `JsonNode` objects that can be navigated like a map or a list. This is useful when the structure of the data is not known ahead of time or when only a few fields need to be read. The third approach is data binding, which is probably the most popular. Data binding maps JSON directly to Java objects using the `ObjectMapper` class, so a developer can write a Java class, call `readValue`, and get back a fully populated instance without having to parse anything manually. The reverse works the same way with `writeValueAsString`.

Jackson also provides a rich set of annotations that control how classes are serialized and deserialized. Annotations like `@JsonProperty` let developers rename fields when converting between JSON and Java, `@JsonIgnore` skips fields that should not be included in the output, and `@JsonFormat` controls how dates and other complex types are formatted. There is also strong support for generics, polymorphism, and nested objects, which makes Jackson flexible enough to handle almost any JSON structure a real application might run into. Beyond the core features, Jackson has official modules for working with Java 8 date and time types, Kotlin classes, Scala types, and reactive streams, which is part of why the library has stayed relevant across so many different Java ecosystems.

Options Available in the API

The Jackson project is split into several modules so that developers only have to include what they actually need. The three core modules are `jackson-core`, which contains the low level streaming API, `jackson-annotations`, which holds the annotation classes used to customize serialization, and `jackson-databind`, which contains the `ObjectMapper` and all of the data binding logic. `Jackson-databind` depends on the other two, so including it also brings them in

automatically. There are also optional modules like `jackson-dataformat-xml` for XML support, `jackson-dataformat-yaml` for YAML, and `jackson-datatype-jsr310` for Java 8 time types. This modular design keeps the core library small and lets developers add features as needed. Configuration happens through the `ObjectMapper` class, which offers dozens of settings for controlling things like whether unknown fields should cause errors, how dates should be formatted, and whether output should be pretty printed or minified.

Download Location

The Jackson JAR files can be downloaded directly from Maven Central at <https://mvnrepository.com/artifact/com.fasterxml.jackson.core>. The three required JARs for basic use are `jackson-core`, `jackson-annotations`, and `jackson-databind`, all three of which are included in the ZIP file submitted alongside this paper. The project source code and release notes are available on GitHub at <https://github.com/FasterXML/jackson>.

Conclusion

Jackson is a strong default choice for any Java project that needs to work with JSON. Its three parsing approaches cover everything from quick lookups to high performance streaming, and its annotation system makes it easy to control exactly how data moves between Java objects and JSON. The fact that it is the default library behind Spring Boot and so many other frameworks means that learning Jackson pays off well beyond any single project.

References

- FasterXML. (2026). Jackson Project Home. GitHub. <https://github.com/FasterXML/jackson>
- FasterXML. (2026). Jackson Databind Documentation. <https://github.com/FasterXML/jackson-databind>
- Baeldung. (2026). Jackson JSON Tutorial. <https://www.baeldung.com/jackson>
- Liang, Y. D. (2019). Introduction to Java Programming and Data Structures (12th ed.). Pearson.